

TOPICAL REVIEW • OPEN ACCESS

Difficulties and approaches in enabling learning-in-memory using crossbar arrays of memristors

To cite this article: Wei Wang *et al* 2024 *Neuromorph. Comput. Eng.* **4** 032002

View the [article online](#) for updates and enhancements.

You may also like

- [Roadmap on emerging hardware and technology for machine learning](#)
Karl Berggren, Qiangfei Xia, Konstantin K Likharev et al.
- [The viability of analog-based accelerators for neuromorphic computing: a survey](#)
Mirembe Musisi-Nkambwe, Sahra Afshari, Hugh Barnaby et al.
- [Excellent nonlinearity of a selection device based on anti-series connected Zener diodes for ultrahigh-density bipolar RRAM arrays](#)
Yingtao Li, Rongrong Li, Liping Fu et al.



TOPICAL REVIEW

OPEN ACCESS

RECEIVED
28 December 2023REVISED
29 June 2024ACCEPTED FOR PUBLICATION
24 July 2024PUBLISHED
1 August 2024

Original content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](#).

Any further distribution
of this work must
maintain attribution to
the author(s) and the title
of the work, journal
citation and DOI.



Difficulties and approaches in enabling learning-in-memory using crossbar arrays of memristors

Wei Wang^{1,*} , Yang Li¹ and Ming Wang²¹ Peng Cheng Laboratory, Shenzhen 518000, People's Republic of China² Frontier Institute of Chip and System, State Key Laboratory of Integrated Chips and Systems, Zhangjiang Fudan International Innovation Center, Fudan University, Shanghai 200433, People's Republic of China

* Author to whom any correspondence should be addressed.

E-mail: wangwei@pcl.ac.cn**Keywords:** learning-in-memory, memristive crossbar, online learning

Abstract

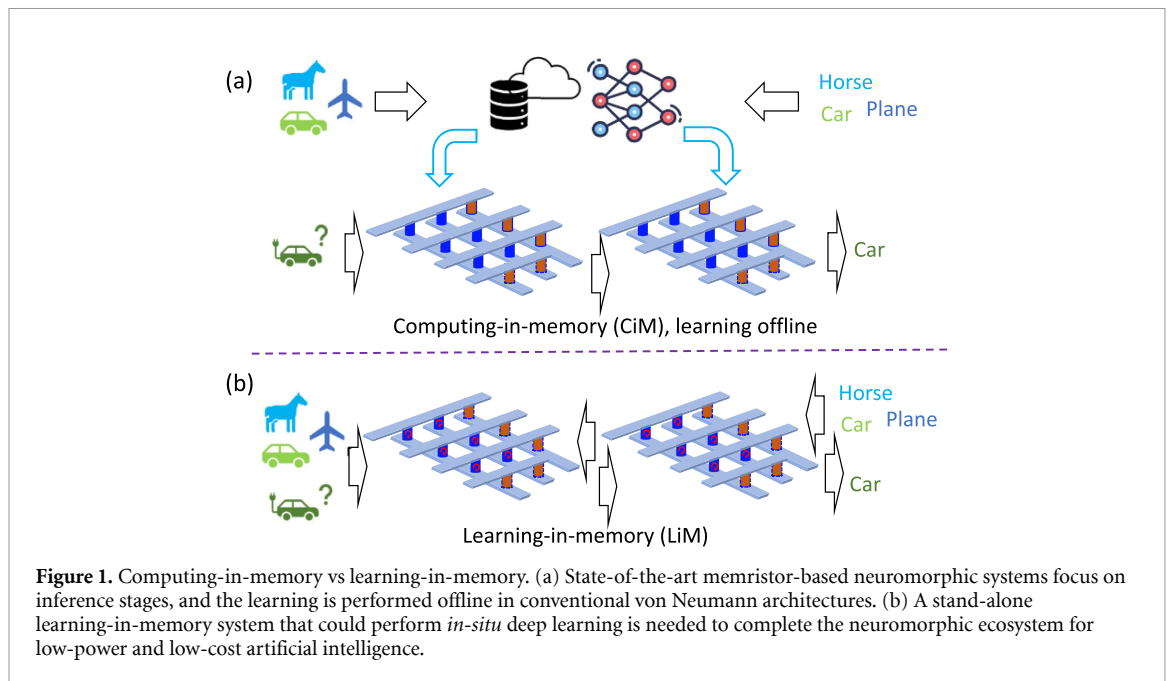
Crossbar arrays of memristors are promising to accelerate the deep learning algorithm as a non-von-Neumann architecture, where the computation happens at the location of the memory. The computations are parallelly conducted employing the basic physical laws. However, current research works mainly focus on the offline training of deep neural networks, i.e. only the information forwarding is accelerated by the crossbar array. Two other essential operations, i.e. error backpropagation and weight update, are mostly simulated and coordinated by a conventional computer in von Neumann architecture, respectively. Several different *in situ* learning schemes incorporating error backpropagation and/or weight updates have been proposed and investigated through neuromorphic simulation. Nevertheless, they met the issues of non-ideal synaptic behaviors of the memristors and the complexities of the neural circuits surrounding crossbar arrays. Here we review the difficulties and approaches in implementing the error backpropagation and weight update operations for online training or in-memory learning that are adapted to noisy and non-ideal memristors. We hope this work will be beneficial for the development of open neuromorphic simulation tools for learning-in-memory systems, and eventually for the hardware implementation of such as system.

1. From computing-in-memory to learning-in-memory

A crossbar array of analog programmable resistors (or memristive devices, or memristors), in principle, can accelerate all the three elementary neural network operations: information forwarding, error backpropagation, and weight update [1, 2]. By coordinating the applied read voltages and measurements of the output currents, the vector-matrix multiplications for information forwarding and error backpropagation can be done in one step [3, 4]. By properly applying write pulses on the crossbars, the vector-vector outer production & accumulation can also be conducted parallelly in all synaptic devices [1]. The benefits of deep neural network acceleration with memristive crossbar arrays are threefold: (i) analog computing is more efficient than digital computing; (ii) data transfer is minimized; (iii) the training and the inference happen in the same hardware.

Unfortunately, most of the recent works focus on the demonstration of the neural network acceleration for inference on edge applications which only consists of the information forwarding process [5–12]. The acceleration of the error backpropagation has been proposed but has been seldomly demonstrated. The *in-situ* weight update process was either performed in the offline training scheme [13–15] or performed one by one and coordinated by external closed-loop control circuits [16, 17].

In inference-only neural networks (figure 1(a)), the neural network parameters are obtained elsewhere, for instance, in a von Neumann architected computing system. That is, the training or learning of the neural network is performed in another computing system. This scheme is usually referred to as offline



training or offline learning. The learned neural network parameters, i.e. the weights, are directly written or transferred to the conductance of memristive devices.

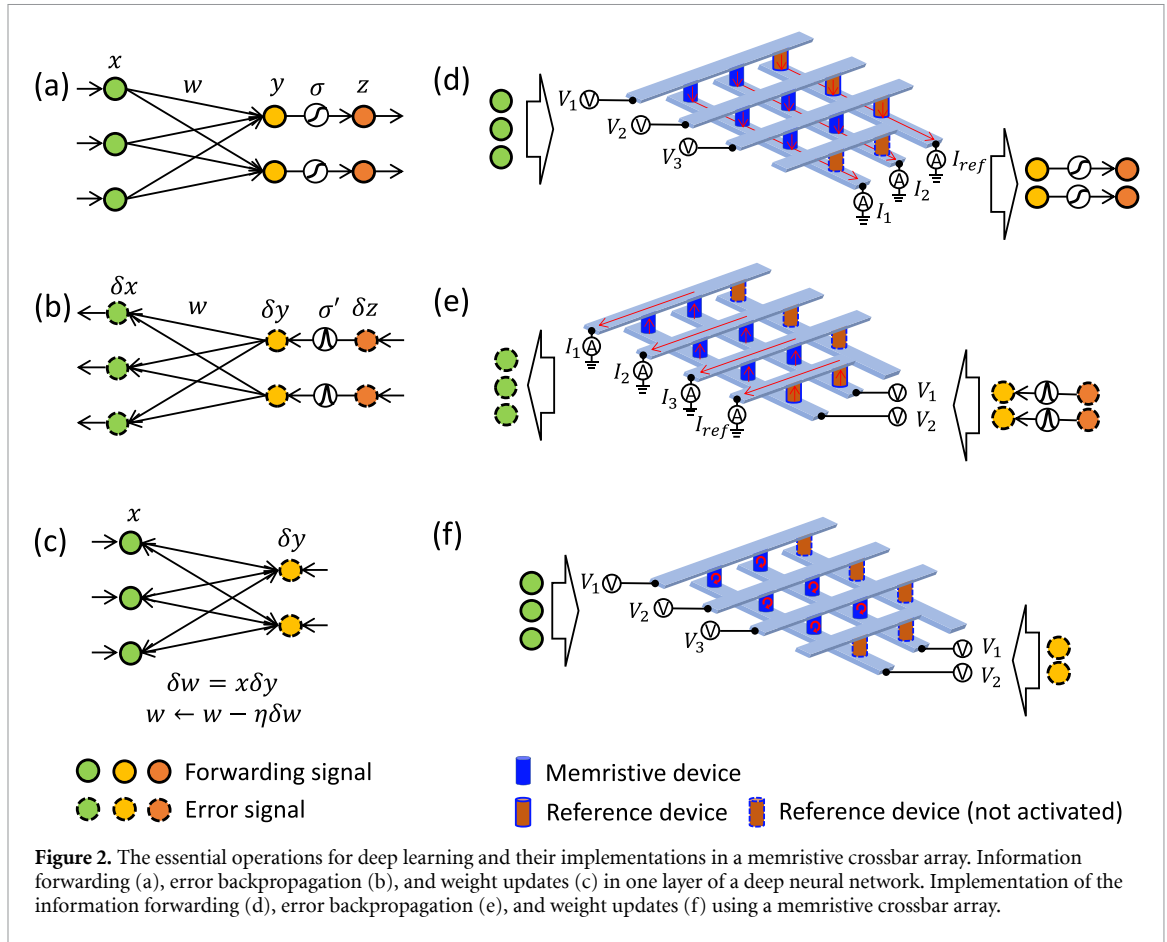
During the inference stage, the memristive devices act as the memory for the weights and, simultaneously, process the forwarding information. The neural network weights are kept in the non-volatile states of the memristive devices; thus, no other volatile or non-volatile memory units are needed, nor the shuttling of weight parameters from the separated memory units as in von Neumann architecture. The information to be processed is forwarded in the pipeline of the end-to-end connected memristive arrays. The non-volatile storage of the neural network parameters and the in-field neural network computation are superior for low footprint and low energy consumption than the von Neumann architecture. Thus, it finds a niche for edge computation. This strategy of using non-volatile memristive devices both for memory and neural processing is commonly referred to as computing-in-memory (CiM). The CiM does not fully exploit the benefits of the memristive crossbar array. Nevertheless, it paved the way for a more advanced learning-in-memory system.

In another aspect, the human brain not only computes in memory but also learns in memory (LiM). The learning is conducted without the coordination or supervision of a von Neumann architecture system. Thus it is of great interest to enable online training or online learning, which refers to the scheme that conducts all three essential operations of a deep neural network in memristive crossbar arrays [18]. This means that there will be no other separate computers of von Neumann architecture involved in the memristive learning system (figure 1(b)). After the training procedure was finished, the same hardware could be employed to conduct the inference task, where only signal forwarding operations are needed.

While there are hybrid training approaches between the offline training and the online training, they are not the ultimate solution. During the hybrid training, the memristive array were used for the signal forwarding, while the error backpropagation and weight gradient calculations were conducted on a computer. The weight updates are performed in the memristive array after a single batch of training samples is processed [19–21].

To enable the LiM schemes, efficient error backpropagation using the crossbar array of memristive devices and appropriate weight updates methods are needed. Here, we review the difficulties in (i) implementing error backpropagation (especially in convolutional layers) and in (ii) updating weights of memristive crossbars with the existence of various non-idealities of synaptic behaviors. Various approaches that have been proposed and should be further investigated to address these difficulties are discussed.

We think the summarization of these approaches and difficulties is a good reference for the neuromorphic simulations work and eventually benefits the development of neuromorphic hardware using crossbar arrays of memristors. We hope this work will bridge the gap between the device engineers who are struggling to develop an ideal synaptic device and neural network algorithmists who are assuming that ideal devices are right at hand. The close of this gap could push forward the information processing system paradigm from CiM to LiM, aiming at a standalone non-von-Neumann computing system.



In this review paper, we first compared the LiM scheme with CiM (section 1). The benefits and additional operations for the LiM scheme are stated. The three elementary operations of deep learning that are needed for completing the LiM tasks are then discussed (section 2). The technologies for the acceleration of information forwarding used for both CIM and LiM are reviewed in section 3. The difficulties and possible approaches to accelerate the error backpropagation for LiM are introduced in section 4. Section 5 reviews the difficulties and approaches regarding weight updates with non-ideal artificial synapses. Section 6 outlooks the future works and proposals.

2. Elementary operations of deep learning

Deep learning, the powerful neural network algorithm for today's artificial intelligence, consists of three essential operations within each layer of the neural network: the information forwarding, the error backpropagation, and the weight update, as illustrated in figure 2(a)–(c), respectively. These three operations correspond to a vector-matrix multiplication, a reciprocal version of the vector-matrix multiplication, and a vector–vector outer production & accumulation, respectively, resulting in huge demands in computational resources in computing systems of von Neumann architecture. Here we use the fully connected layer to illustrate the basic algorithm of neural network operations. More issues existing in the hardware implementation of other typical layers, such as convolutional layers, will be covered in subsequent sections.

2.1. Information forwarding

The information forwarding of a fully connected layer in a deep neural network is conducted by first multiplying the input vector $\mathbf{x}$ with the weight matrix $\mathbf{w}$ (i.e. vector-matrix multiplication), and then processing the production vector $\mathbf{y} = \mathbf{x}\mathbf{w}$ through an activation function $\sigma(\cdot)$ (figure 2(a)). The output of the activation function, vector $\mathbf{z} = \sigma(\mathbf{y})$, is the input of the next layer.

If we assume the neural network layer has m (the size of the input vector $\mathbf{x}$) input neurons and n (the size of the output vector $\mathbf{y}$ and $\mathbf{z}$) output neurons, the weight matrix should have a size of m -by- n . The

vector-matrix multiplication, $\mathbf{y} = \mathbf{x}\mathbf{w}$, can be detailed as,

$$y_j = \sum_{i=1}^m x_i w_{ij}, \quad j = 1, \dots, n. \quad (1)$$

The activation function is applied to the output vector $\mathbf{y}$ element-wisely, that is,

$$z_j = \sigma(y_j), \quad j = 1, \dots, n. \quad (2)$$

2.2. Error backpropagation

The error backpropagation (figure 2(b)) works in a reversed direction compared with the information forwarding. The error signal from the post layer, $\delta\mathbf{z}$, is first multiplied by the derivative of the activation function, $\delta\mathbf{y} = \delta\mathbf{z} \sigma'(\mathbf{y})$, which can be denoted in element-wise form,

$$\delta y_j = \delta z_j \sigma'(y_j), \quad j = 1, \dots, n. \quad (3)$$

Then the production is multiplied by the weight matrix, $\delta\mathbf{x} = \delta\mathbf{y} \mathbf{w}^T$,

$$\delta x_i = \sum_{j=1}^n \delta y_j w_{ij}, \quad i = 1, \dots, m \quad (4)$$

which is another vector-matrix multiplication again.

2.3. Weight update

The weight can be updated after the forwarding information $\mathbf{x}$ and the backpropagating error $\delta\mathbf{y}$ are both available on the two sides of the weight matrix (figure 2(c)). The gradient of the weight matrix with respect to the loss function of the neural network is $\delta\mathbf{w} = \mathbf{x}\delta\mathbf{y}$, which is,

$$\delta w_{ij} = x_i \delta y_j, \quad i = 1, \dots, m, \quad j = 1, \dots, n. \quad (5)$$

The weight is then updated according to the gradient descent rule to minimize the loss function of the neural network, $\mathbf{w} = \mathbf{w} - \eta \delta\mathbf{w}$, or,

$$w_{ij} \leftarrow w_{ij} - \eta \delta w_{ij}, \quad (6)$$

where η is the learning rate, which is usually a small number, e.g. $0.001 \sim 0.1$.

2.4. The computational load of deep learning

Since all the element values in the neural network operations are usually represented by floating-point numbers, completing the calculation denoted by equation (1) in a computer of von Neumann architecture, needs to conduct the multiplications of two floating-point numbers for $m \times n$ times and additions (or accumulation) of two floating-point numbers for approximate $m \times n$ times. Each pair of multiplication and accumulation is called one multiply & accumulate operation (MAC). Thus, conducting the vector-matrix multiplication needs to perform $2 \times m \times n$ floating-point operations (FLOPs) or $m \times n$ MACs. The error backpropagation needs the same number of MACs for the vector-matrix multiplication (equation (4)). Combining equations (5) and (6), we see that, for the weight update process, we need $m \times n$ times of multiplications of three scalars (x_i , δy_j , and η) and $m \times n$ times of accumulations. In total, the single neural network layer needs $3 \times m \times n$ MACs for training the neural network with one training sample.

The activation function also requires arithmetic calculations. The number of calculation steps depends on the choice of the activation function. For instance, rectified linear unit (ReLU) requires fewer calculation steps than sigmoid activation functions. Basically, the number of calculation steps is proportional to the number of neurons as denoted by equations (2) and (3).

For a typical three-layer fully connected neural network with the size of 784–500–250–10 for the recognition of handwritten digits in the modified National Institute of Technology (MNIST) dataset, the number of weight elements or parameters is 519, 500 ($784 \times 500 + 500 \times 250 + 250 \times 10$). Roughly, approximate 1.5 million ($519, 500 \times 3$) MACs need to be performed for training an image without counting the calculations for the neural activations (equations (2) and (3)). The training set in the MNIST datasets contains 60 000 images and successful training of the neural network usually takes about hundreds of epochs, which needs totally 9×10^{12} MACs for 100 training epochs.

For a convolutional layer in deep neural networks, where weight connections are spatially shared, and for a recurrent layer, where weight connections are temporally shared, the core computations are still the

vector-matrix multiplication between the input vector and the weight kernels. Although the calculations of computation loads would be more complex, the error backpropagation and the weight update processes share similar numbers of MACs with the information forwarding process.

In modern deep neural networks, the number of parameters and the required FLOPS are too large to be handled efficiently. For instance, ResNet-50, a 50-layer residual network has 25.6 million parameters and needs to perform 4.1 billion MACs for forwarding a single input image [22, 23]. Thus, training of a single image roughly takes 12.3 billion MACs. The ResNet-50 was trained with 1.28 million images from the ImageNet 2012 classification dataset and 128 epochs [22], counting up to 9.4×10^{18} MACs. AlphaGoZero [24] and AlphaZero [25] developed by DeepMind increase the computation demands, i.e. the cost in dollars, by roughly 3–4 orders of magnitude compared with ResNet-50 [26, 27].

These operations could be performed sequentially or parallelly in the floating-point units of a central processing unit or a graphics processing unit. First, the complementary metal-oxide-semiconductor circuit performing the high-precision MACs is complex and consumes large static and dynamic energy. Second, the data of the forwarding information, the backpropagation errors, and the neural network parameters are stored in off-chip memory. They are shuttled between the off-chip memory and the on-chip floating-point units, which consumes much more energy than the MAC operations themselves and results in high latency. This is known as the memory wall issue.

For the first issue, there are neural network quantization methods to reduce the precisions of the forwarding information and the parameters, resulting in low-precision or integer-based neural networks [28, 29]. The second issue can be addressed by moving the off-chip memory close to the processing units and reforming the data flow, as can be seen in multiple application-specific integrated circuits chips specified for artificial intelligence, such as Google's tensor processing unit [28], Cambrian's DaDianNao [30], Microsoft's Azure Maia AI Accelerator³, and Amazon's Trainium2⁴.

3. Acceleration of information forwarding with memristive crossbar arrays

The neuromorphic approach based on crossbar arrays of analog programmable resistors or artificial synapses solves the above issues in a radically different way.

3.1. Forward vector-matrix multiplication

The connection weights $\mathbf{w} = \{w_{ij}, i = 1, \dots, m, j = 1, \dots, n\}$ in figure 2(a), i.e. the neural network parameters can be stored as the conductance of a crossbar array of analog programmable resistors $\mathbf{G} = \{G_{ij}, i = 1, \dots, m, j = 1, \dots, n\}$, as shown in figure 2(d). For simplicity, we use $m = 3$ and $n = 2$, in both figures 2(a) and (d) for illustration. Since the conductance of the resistors is always positive, to represent negative weight information using positive conductances, we use a column of reference resistors of the same conductance G_{ref} . The mapping between the signed connection weights and the conductance of the resistors can be expressed as,

$$G_{ij} = G_0 w_{ij} + G_{\text{ref}}, \quad (7)$$

where G_0 is the scaling factor.

The input vector $\mathbf{x} = (x_1, \dots, x_i, \dots, x_n)$ was converted to a vector of analog voltage, $\mathbf{V}_x = (V_{x1}, \dots, V_{xi}, \dots, V_{xm})$ with the scaling factor, V_0 ,

$$V_{xi} = V_0 x_i. \quad (8)$$

The analog voltage vector is then applied to the top bars of the crossbar array. The bottom bars of the crossbar array are grounded and the currents to the ground are measured, which are,

$$I_j = \sum_{i=1}^m V_{xi} G_{ij}, \quad (9)$$

for each regular column (bottom bar), and

$$I_{\text{ref}} = \sum_{i=1}^m V_{xi} G_{\text{ref}}, \quad (10)$$

³ <https://news.microsoft.com/source/features/ai/in-house-chips-silicon-to-service-to-meet-ai-demand/>.

⁴ <https://aws.amazon.com/machine-learning/trainium/>.

for the reference column. Subtract equation (10) from equation (9) and combine with equations (7) and (8), we obtained,

$$I_j - I_{\text{ref}} = \sum_{i=1}^m V_{xi} (G_{ij} - G_{\text{ref}}) = V_0 G_0 \sum_{i=1}^m x_i w_{ij}. \quad (11)$$

Comparing equation (11) with equation (1), we would find that the vector-matrix multiplication has been completed, if we write

$$y_j = \frac{I_j - I_{\text{ref}}}{V_0 G_0}. \quad (12)$$

The multiplication is conducted by Ohm's law inherently, i.e. the current through the resistor is the product of the applied voltage and the conductance of the resistor, while the accumulation or addition is conducted by Kirchhoff's current law, i.e. the currents through the resistors in the same column are merged in the bottom bar. Instead of performing the MACs in the vector-matrix multiplication one-by-one as denoted by equation (1), all MACs are performing parallelly in one step governed by Ohm's law and Kirchhoff's current law as denoted by equation (11).

To complete the MAC operation by Ohm's law and Kirchhoff's current law, accurate voltages (equation (8)) need to be applied to the top bars of the array and the currents from the bottom bars (equations (9) and (10)) need to be accurately sensed. The application of accurate voltages generally requires digital-to-analog convertors (DACs). Accurate sensing of current generally requires analog-to-digital convertors (ADCs) along with other auxiliaries, such as, transimpedance amplifier (TIAs) sampling and hold circuits [31]. In this way, the MAC operations are conducted in analog domain, and the neuron functionalities (such as activation function) are performed in digital domain. However, these peripheral circuits complex the MAC operations and are costly in both area and energy consumption.

Other than conducting the MAC operation using voltage amplitude to represent the input value and current amplitude to represent the output value, respectively, it is also possible to perform the MAC operation in time domain [32, 33]. That is, the input value is represented by the pulse number or pulse duration with fixed amplitude, and the output value is obtained by integrating the current from the bottom bars. Still, complex peripheral circuits around the crossbar array are need.

Alternatively, the neuron functionalities can be performed in analog domain. For instance, Kiani *et al* reports a fully hardware-based memristive multilayer neural network with the ReLU activation function is implemented by analog circuits [34]. The current output of the preceding layer is converted to voltages and rectified by a hybrid circuit of TIAs and rectifying diodes. This arrangement removes the needs of the calculation of activation function in the digital domain which is usually conducted by an arithmetic logic unit.

The accuracy of MAC operation is affected by several non-idealities. The inaccuracy of weight-conductance mapping (equation (7)), when writing the weight to the conductance of memristive devices. Read noise and conductance drift in long term could also degrade the accuracy of MAC calculation [35, 36]. The truncation error of DACs and ADCs with limited bit width is another source of inaccuracy for MAC operation [37]. However, the MAC calculation accuracy is not the only factor that affects the performance of the neural network. For inference-only neuromorphic system, the degradation of the neural network performance due to the inaccuracy of MAC operation can be compensated by hardware-aware training [13]. For hybrid and online training, the neural network could usually be adapted to minimize the effect of these non-idealities [19].

3.2. Information forwarding in convolutional layers

Information forwarding in a convolutional layer is implemented by unrolling the two-dimensional image patches and weight kernels into single-column vectors [16, 38]. Different kernels are arranged together to form a matrix which could be implemented by a memristive crossbar array.

Figure 3(a) illustrates the operation of a typical convolutional layer with 3 input feature maps, 2 output feature maps, and 3-by-2 convolutional kernels. Each kernel has a size of 2-by-2. The first group of kernels connects the three patches of input feature maps into one pixel of the first output feature map. The second group of kernels connects the same three patches of input feature maps into one pixel of the second output feature map.

Thus, the weights in the convolutional kernels can be mapped to a crossbar array of memristors, with the size of 12-by-2 as shown in figure 3(b). Weights in the same kernel are unrolled to the same column. For simplicity, the reference column for representing sign weights is depressed in the illustrative figure.

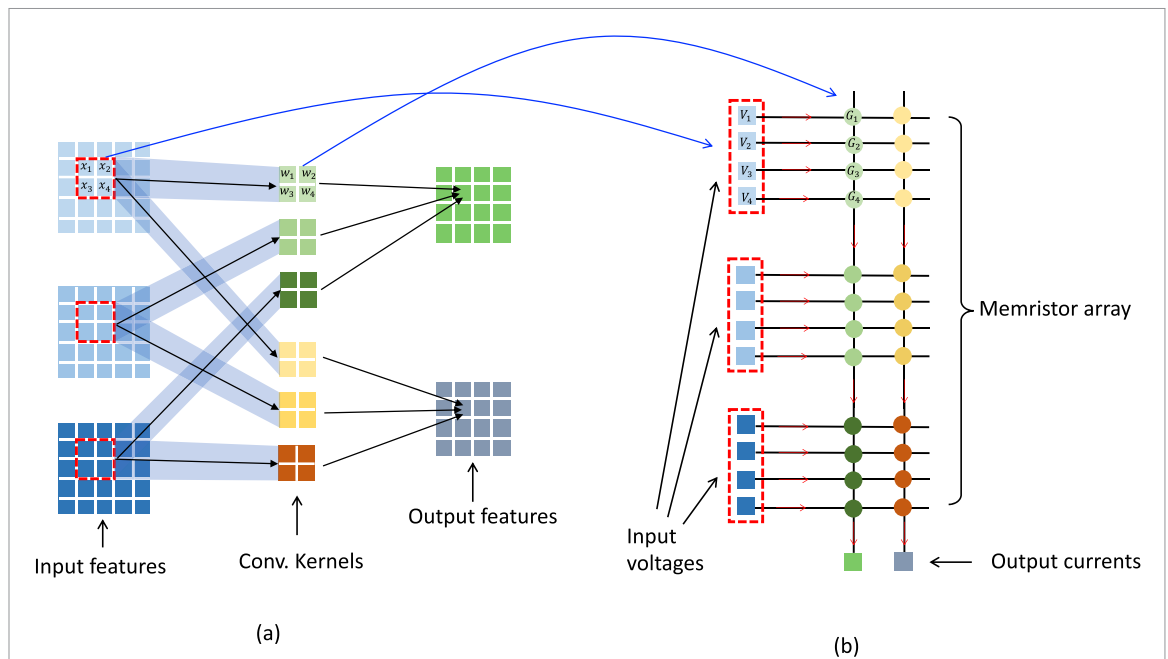


Figure 3. Information forwarding through convolutional layers. (a) Illustration of the convolutional operations between input feature maps and convolutional kernels for resulting in output feature maps in a typical convolutional layer. (b) Mapping of the convolutional layer into vector-matrix multiplication of memristor array.

Input voltage vectors that are converted from image patches are fed to the crossbar array to generate output currents as the elements of the output feature maps. When sliding the convolutional windows in the input feature maps, input voltage vectors that are from sliding image patches are fed to the crossbar array sequentially. This means that the spatially shared weight kernels are actually temporally shared for sliding windows, which increases the latency of information forwarding in the convolutional layer. It is also possible to duplicate the memristor array such that different input image patches can be convoluted parallelly. A trade-off between memristor array size, i.e. the footprint of the memristive acceleration module, and the latency should be made.

4. Error backpropagation through memristive crossbar array

4.1. Backward vector-matrix multiplication

For training a deep neural network on a conventional computer, the error backpropagation consumes the same amount of computation resources as the signal forwarding. The implementation of error backpropagation of a fully connected layer with a memristive crossbar array seems straightforward if only looking at figures 2(b) and (e). However, several issues prevented the realization.

The implementation of the error backpropagation on the memristive crossbar array is similar to that of the information forwarding, except that: (i) the vector-matrix multiplication direction is reversed, i.e. the input voltage vector is applied to the bottom bars and the current output vector is measured in the top bars; (ii) a row of reference resistors instead of a column of reference resistors is needed.

A duplicated set of voltage sources (e.g. DACs) and current meters (e.g. TIAs; sampling and hold circuits; ADCs) is needed. It is also possible to reroute the voltage sources and current meters that are used for forwarding VMM to conduct the task of backward VMM. In fact, in the forwarding VMM, the voltage sources and current meters usually are not exclusively assigned to each row and column bar of the memristive array. In most reported works, multiple row bars share one voltage source and multiple column bars share one current meter, to reduce the overall footprint of the peripheral circuit. This arrangement, however, reduces the throughput of the VMM operations. In addition, the error signals are small, which need more accurate voltage sources tuning and currents sensing for the backward VMM.

4.2. Error backpropagation in a convolutional layer

The forwarding VMM and backward VMM in the same crossbar array in figures 2(b) and (e) are not valid in convolutional layers, where spatially shared weights are needed. This is because the deconvolution operation needs to reshape both the weight kernels and the convolutional patches.

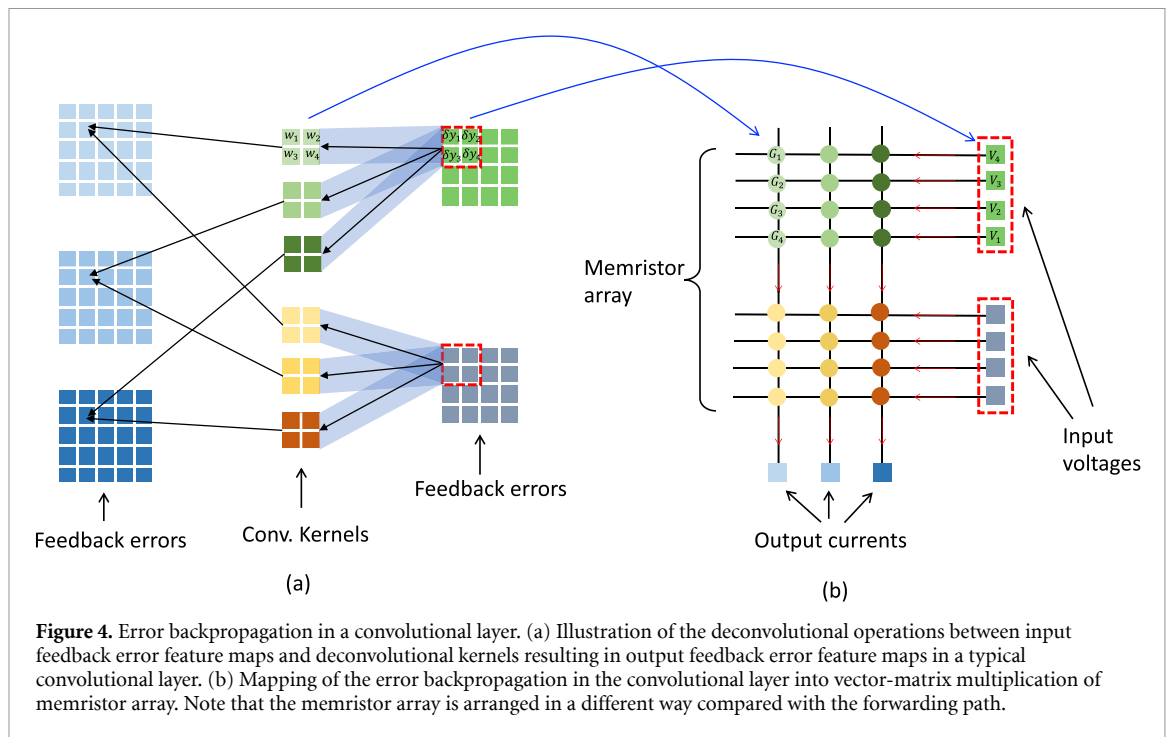


Figure 4. Error backpropagation in a convolutional layer. (a) Illustration of the deconvolutional operations between input feedback error feature maps and deconvolutional kernels resulting in output feedback error feature maps in a typical convolutional layer. (b) Mapping of the error backpropagation in the convolutional layer into vector-matrix multiplication of memristor array. Note that the memristor array is arranged in a different way compared with the forwarding path.

Figure 4 illustrates the error backpropagation of the convolutional layer shown in figure 3. The first kernels in all kernel groups connect the patches of input feedback error maps with one pixel of the first output feedback error maps. The second kernels in all kernel groups connect the patches of input feedback error maps with one pixel of the second output feedback error maps, and so on. Thus, the weights in the deconvolutional kernels can be mapped to a crossbar array of memristors, with the size of 8-by-3 as shown in figure 4(b). Weights of certain kernels that are at the same location of each kernel group are unrolled to the same column.

It should be noted that the memristor array for error backpropagation is in a different form compared with the array for information forwarding. This requires the reconnection of wire bars when switching between forward and backward passes of the convolutional layer operations. Alternatively, one can also use separated crossbar arrays of memristors for the forward and backward paths. However, the conductance of elementary memristors should be updated at the same pace.

4.3. Possible ways of by-passing the error backpropagation

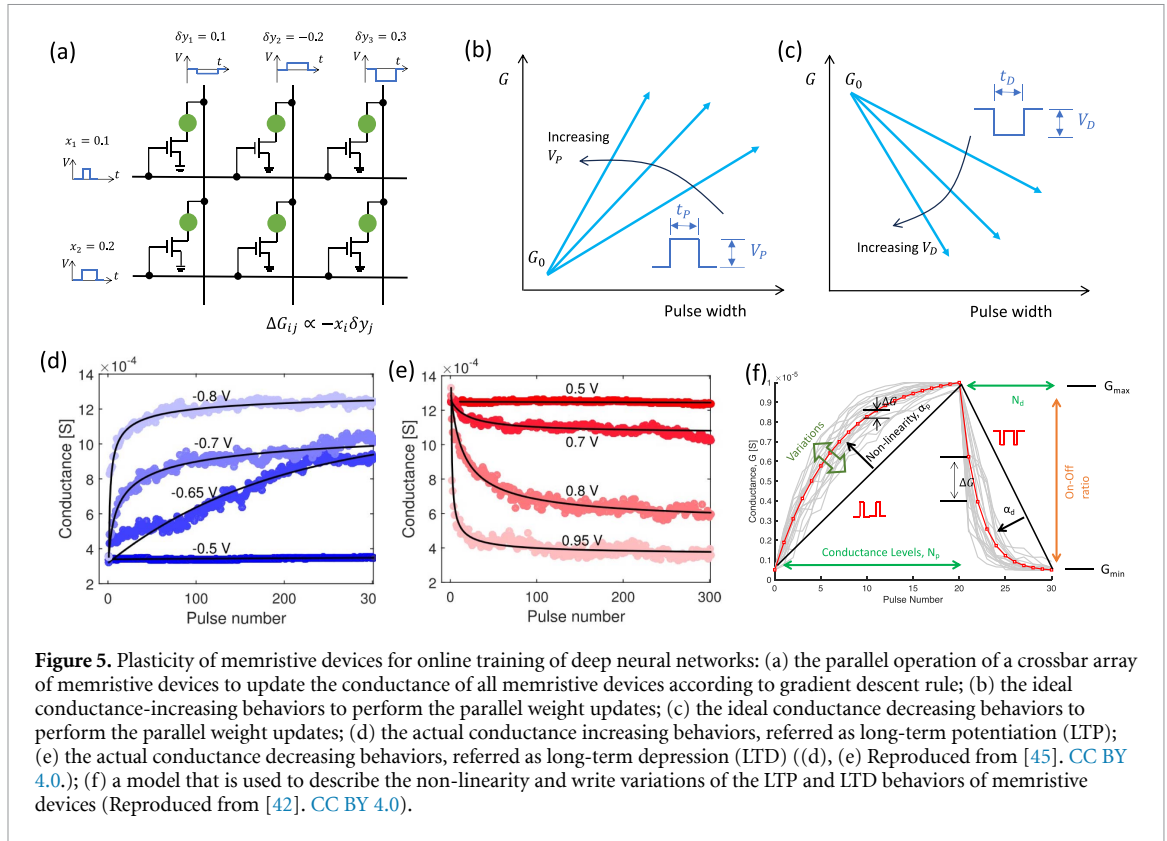
The error backpropagation algorithm is a mathematically elegant learning algorithm, however, it is not supported by physiological observations [39]. This is consistent with the situation in electrical circuit hardware. In other words, it is hard to backpropagate the learning errors in the neuromorphic approach. Novel learning methods that are more biologically plausible, and thus may also be more suitable for the neuromorphic implementation, should be explored.

Autoencoder networks based on restricted Boltzmann machines and deep belief networks [40, 41] remove the need for error backpropagation for learning deep neural networks. Instead of error backpropagation, contrastive divergence and greedy learning methods are used for single-layer learning and multiple-layer learning, respectively [42].

There is also another more radically different and biologically plausible way to conduct the credit assignment task for deep learning, named feedback alignment [43]. In the feedback alignment method, the backpropagation of errors is not necessary through the symmetric connections of the forwarding path. The implementation of this novel learning scheme is, however, an open issue to be explored by the research community of neuromorphic computing.

5. Weight updates in a memristive crossbar array

Weight update corresponds to the parameter adjustments in a neural network, which is essential for the 'learning' ability of the neural network. Weight update happens when the backward errors meet the forward signals (figures 2(c) and (f)).



5.1. The ideal case

The weight update ideally can also be realized by the crossbar array in one step by parallelly programming all the devices. As shown in figure 2(f), it is possible to engineer the electrical excitations applied to the top bars and the bottom bars, such that the conductance changes would be proportional to the amplitude of the product of the forwarding information and the backpropagating error.

For instance, Agarwal *et al* [1, 44] proposed to translate the forwarding information x into the width of positive voltage pulses and the backpropagating error δy into pulse amplitude of negative voltage pulses. This arrangement will result in an overlap of the voltage pulses from the top bar and the voltage pulses from the bottom bar on each memristive device in the crossbar array. Figure 5(a) displays a modified version of Agarwal's proposal on a crossbar array of one-transistor/one-resistor memristors. Each memristive device (G_{ij}) will undergo a voltage pulse with the pulse width proportional to the forwarding information x_i and the pulse amplitude proportional to the backpropagation error δy_j .

In an ideal case, as shown in figures 5(b) and (c), the conductance change of the memristive devices should be proportional to the strength of the write excitation, resulting in,

$$\Delta G_{ij} \propto -x_i \delta y_j, \quad (13)$$

and,

$$G_{ij} \leftarrow G_{ij} + \Delta G_{ij} \quad (14)$$

which complete the weight update denoted by equations (5) and (6). The learning rate in equation (6) is defined by the plasticity of the memristive device.

However, the ideal memristive plasticity shown in figures 5(b) and (c), is not available in practical electronic devices. Biological synapses do not show such ideal plasticity either. In other words, neither memristive devices nor biological synapses can support the parallel weight update rules denoted by equations (13) and (14) for deep learning. Thus, modification of the weight update rules should be made both to enable the online deep learning of memristive deep neural networks and to shed light on possible learning mechanisms of biological systems.

5.2. Plasticity of memristive devices

The non-idealities of the real memristive devices that prevent the application of the ideal weight update scheme include non-linearity, write variations, limited conductance states, limited on/off ratio, and limited

retention time [46]. Among them, the non-linearity and write variations are the most troublesome issues. Figures 5(d) and (e) show typical conductance changes under continuous identical pulses which aims to increase and decrease the conductance of the memristive device, respectively [45]. These curves are often referred to as long-term potentiation (LTP) and long-term depression (LTD) behaviors of the artificial synapses, respectively.

The LTP and LTD behaviors of memristive devices are usually measured under continuous identical pulses. This is because the conductance changes at various write pulse widths and pulse amplitudes can be generally projected through the conductance tuning curves obtained from measured LTP and LTD behaviors under identical pulses. On the other hand, however, tuning the conductance by identical pulses (fixed pulse width and pulse amplitude for LTP and LTD) is friendly for hardware implementations of the *in-situ* weight update during online learning.

From the actual LTP and LTD behaviors of memristive devices, we can see that the conductance of memristive devices is bounded and tends to be saturated when the conductance is close to the conductance limits at either tune direction. This is the natural behavior of any physical device since the internal changes of the device that are induced by external stimuli will naturally decrease when the state of the device approximates its physical bounds. The write variations, including cycle-to-cycle variations and device-to-device variations, are also natural behaviors of memristive devices, originating from various resistive switching mechanisms.

A behavior-level model of the conductance changes for the LTP and LTD is usually needed to investigate the deep learning performance under various non-idealities of the memristive devices [46, 47]. The behavior-level model needs to incorporate the non-linear weight update behavior, the limited on/off ratio, limited conductance levels, the asymmetry between LTP and LTD, and write variations, as shown in figure 5(f) [42]. The model is also necessary to validate the various weight update schemes to achieve the LiM goal.

5.3. Weight update schemes

With the existence of the non-idealities of memristive synaptic devices, the parallel weight updating scheme shown in figure 5(a) would deviate significantly from the ideal case. Thus, modified and mostly compromised variants of weight update methods are applied. These compromised versions might (i) be only possible to update synapse weights of a crossbar array of memristive device row-by-row or column-by-column, or even single-device-by-single-device; (ii) need multiple write-and-read iterations to perform one weight update operation of a single memristive device.

5.3.1. Manhattan update rule

In the Manhattan learning rule, the amount of weight changes is disregarded, leaving only the direction of weight changes [33, 48, 49]. The Manhattan rule reads,

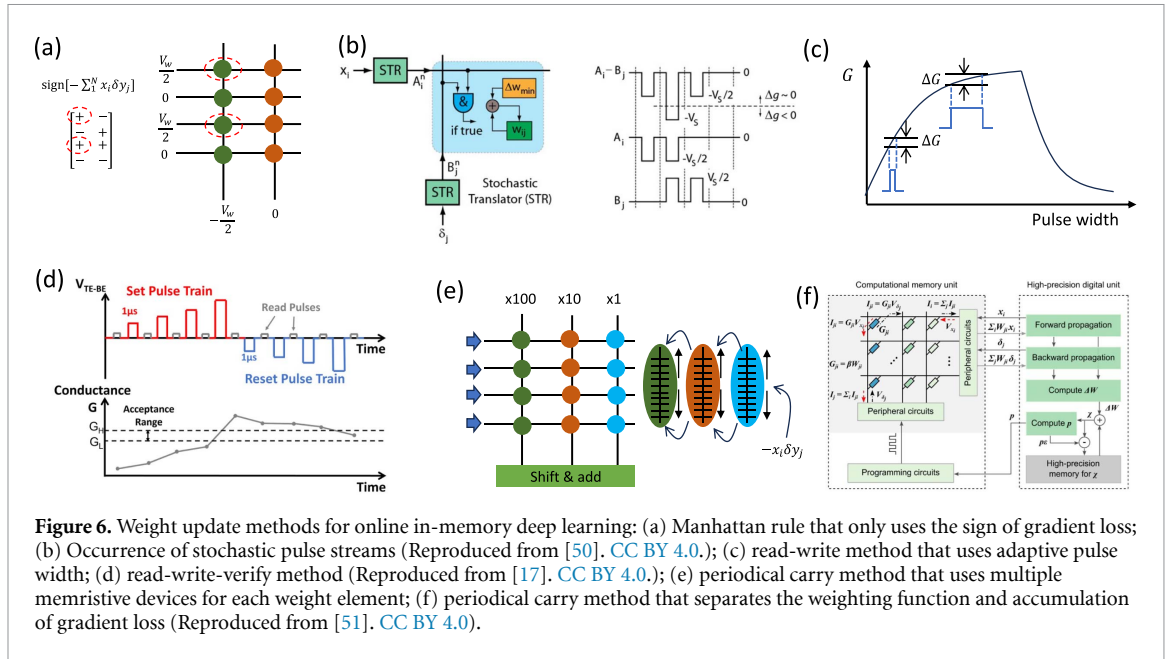
$$\Delta w_{ij} = \eta \text{sign} \left(- \sum_1^N x_i \delta y_j \right) \quad (15)$$

where the learning rate η indicates a uniform weight update step. The summation in equation (15) indicates that the direction of the weight changes can be calculated after a batch of training samples (with the batch size of N) has been processed.

After all the weight change directions are obtained, the conductance of corresponding devices are updated through identical write operations, as shown in figure 6(a) [48]. That is, all devices whose conductance needs to be increased undergo the same write ‘set’ style operation. The learning rate in equation (15) is then defined by the strength of the write operation and the intrinsic behavior of the memristive devices. Vice versa, all devices whose conductance needs to be decreased undergo the same write ‘reset’ style operation.

These ‘set’ style operations and ‘reset’ style operations can be performed column-by-column (or row-by-row) in the memristive crossbar array. For instance, the memristive devices that needed to be potentiated in the first column are parallelly potentiated in a half-write-voltage scheme, that is, half of the write voltage on the column bar and half of the negative write voltage on the row bars of corresponding devices, as shown in figure 6(a).

There are two factors that are responsible for the accuracy loss of online training using the Manhattan learning rule. First, the Manhattan rule itself disregards the magnitude of the gradient of the loss function to the weights. Second, the conductance changes induced by the identical write operations on memristor devices are not uniform as required by the ideal Manhattan learning rule. As a result, the learning ability of the Manhattan learning rule is limited. On the positive side, the conductance of the memristive device is naturally bounded and induces weight limits to the Manhattan rule. In the software version of the



Manhattan rule, however, the weight needs to be monitored to be kept within limits to avoid oscillation and arithmetic overflow [52].

Using the threshold Manhattan learning rule, it is possible to make the learning process more robust by further disregarding the small loss gradient. That is, only the loss gradients that are higher than a threshold will be used for updating the conductance of the memristive device. Since most of the loss gradient is small, the number of write operations on memristive devices is greatly reduced. This method has been demonstrated in training the last layer of deep neural networks [53]. It has not been shown whether the Manhattan learning rule or the threshold Manhattan learning rule could be successfully applied to preceding layers using the loss gradient calculated by feedback errors.

5.3.2. Occurrence of stochastic streams

To increase the learning accuracy, it is essential to keep the magnitude of the gradient loss. While identical write pulses are strongly recommended considering the hardware implementation of the write or weight update circuit, the magnitude of gradient loss can be encoded by the probability of the identical write operations.

Figure 6(b) illustratively shows the design of using the probability of write operations to update a single weight element [50]. The weight update is performed by applying stochastic pulse streams on the two sides of the memristive devices. The probability of the negative half-voltage write pulse on one side is proportional to the forwarding signal x_i , while the probability of the half-write pulse on another side is proportional to the feedback error δy_j . The occurrence of the negative half-voltage write pulse and the half-write pulse, which has the probability of $x_i \delta y_j$, results in the decrease of the conductance changes.

In this method, the pulse streams can be parallelly applied to all row bars and column bars of the memristive array. In addition, the calculation of the gradient loss to each device has already been conducted *in situ*, i.e. the production $x_i \delta y_j$ do not need to be performed externally.

The non-uniform conductance change under identical write pulses would be a major problem of the online training [50, 54]. Bounded conductance ranges, non-linear weight updates, and write variations are the major sources of the degradation of the training accuracy. When all these non-idealities are well suppressed by device engineering, which is not an easy task, near software training accuracy could be obtained.

The Manhattan learning rule and occurrence of stochastic streams are both blind write methods, because they conduct the write operations without caring about the current states of the memristive devices [55].

5.3.3. Read-write

If the weight update behaviors of the memristive devices, which depend on the current state and the strength of excitation, can be well predicted, the non-linear weight update issue could be passed. In the read-write method, for a single weight update, a read operation is needed to determine the current state of the

memristive devices. The device state is updated by applying a write operation whose strength is calculated depending both on the current state of the device and the requested amount of the weight change [19–21].

For instance, when the conductance increase is calculated, the pulse width of potentiation pulses could be adjusted according to the current conductance states and the LTP curve, as shown in figure 6(c). If the current conductance is near the minimal conductance limit, a shorter potentiation pulse should be applied. Otherwise, a longer potentiation pulse is needed. Similar operations are needed for the conductance depression requirement by the learning algorithm.

This weight update method needs one read operation before the write operation to obtain the current conductance of the memristor device and could not be parallelly conducted for all memristive devices in the crossbar array. In addition, variable pulse width is not friendly for hardware implementation of the write circuit.

5.3.4. Read-write-verify

The accuracy of conductance tuning of memristive devices could be greatly improved if the read-write-verify method is employed, which suppresses the non-linear weight update and write variation effects [16, 17, 56]. This method is also called the closed-loop write method.

First, the current conductance of the memristive device is read out and the conductance change is calculated from the learning algorithm. They determine a target conductance of a write and verify loop.

In the loop, if the current conductance is lower or higher than the target conductance, a potentiation or a depression pulse is applied, respectively. Then the new current conductance state is read out to verify if it has been written to a state within a range near the target conductance. Otherwise, the loop is continued. One can use increasing pulse amplitude or uniform pulse amplitude when continuous potentiation or depression pulses are needed in the loop [57].

The closed loop read-write-verify method is an effective, but not an efficient, way to fine-tune the conductance of memristive devices in a crossbar array. The accuracy of the conductance tuning is defined by defined resolution near the target conductance. The higher the resolution, the longer the write and verify loop. The longer write and verify loop, however, would greatly slow the online learning process, since hundreds of write operations are needed in a single loop and it could only be performed device-by-device in the crossbar array. Yao *et al* have demonstrated that this read-write-verify method could tune the conductance of a memristive device to have a 5-bit representation ability (32 levels) [16]. Higher write accuracy could be obtained in principle if a longer write and verify loop is tolerable by the learning process.

5.3.5. Periodical carry

The periodical carry algorithm uses more than one artificial synapse for each weight cell of the weight array. The gradient losses are used to update the low significant weights, and the higher significant weights are periodically updated when detected that the lower significant weights are out of their limits and generate carry signals. Different high-significant and low-significant weight implementation methods have been proposed. Agarwal *et al* proposed to use memristive devices for both high and low significant weights, as shown in figure 6(d) [44]. Ambrogio *et al* proposed to use memristive devices as the high significant weights and capacitor-based artificial synapses as the low significant weights [58–60].

A similar approach, named as ‘tiki-taka’ algorithm [61], has also been proposed. Instead of updating the high significant weights after the low significant weights overflow or reach a threshold, the ‘tiki-taka’ algorithm updates the high significant weights according to a randomly selected small patch of the low significant weights. The high-significant weights have a much lower update frequency than the low-significant weights, which is similar to the periodical carry method.

The periodical carry method is effective in mitigating the troubles induced by the non-idealities of conductance updates. It enables the possibility of parallel weight updates for all synapses in the crossbar array, thus promising efficient online in-memory deep learning.

It is further found that the low significant weights are not necessary to participate in the information forwarding and error backpropagation processes of the neural network [51, 62–64]. Thus, the gradient of the loss to the weight could be accumulated in a separate array outside of the crossbar of memristive devices, as shown in figure 6(f). The separated array could be a high-precision digital memory [51] or digital counters [42] according to the different ways of calculating the gradient of loss to the weight. The separation of the accumulation of the gradient loss and the conductance updates is highly tolerant to various non-idealities of memristive synaptic behaviors with the blind write method and greatly reduces the number of write operations on memristive devices.

6. Discussion and conclusion

To enable *in situ* deep learning in analog memory, innovations from both learning algorithms and synaptic devices are needed.

From the aspect of learning algorithms, the conventional error backpropagation method is not friendly for a crossbar array of memristive devices, especially for layers that are not fully connected, such as convolutional layers. This is consistent with the case in biological LiM systems, where error backpropagation through the same synaptic path in a reversed direction is not supported [39]. Continuous learning algorithm innovations in network topology are intensively needed.

From the side of the synaptic devices, materials and devices engineering toward ideal synaptic behaviors are always required. However, for even quite noisy and nonlinear synaptic behaviors, there are also ways to compensate them using different weight update schemes at various costs, in terms of the complexities and area of the write circuits, the number of write operations, and the energy and the latency of the weight update. Device engineers should avoid claiming high *in situ* deep learning performance without mentioning the exact methods for error backpropagation and weight updates.

Nonlinearity and write variations are the most heinous properties from the point of view of learning algorithms. However, nonlinear weight update is unavoidable in any physical programmable resistors when their whole dynamic ranges are exploited. Blind write methods are the most efficient learning schemes, however, require even more ideal synaptic behaviors. The closed-loop read-write-verify method could tolerate most non-ideal synaptic behaviors, however, it is expensive and should not be preferred for *in situ* deep learning in a practical system. The periodical carry algorithm and its variations show great tolerance to various non-idealities of synaptic behavior of memristive devices and request low complexity of peripheral circuits around the crossbar array. These algorithms are underdeveloped and need further verifications in real hardware systems.

In conclusion, we reviewed the necessary components for deep learning and their difficulties in being implemented and accelerated by the crossbar array of analog programmable resistors. Only the information forwarding process is well studied, whereas the error backpropagations and weight updates do not have a clear path yet. We compare techniques that have been proposed to incorporate all the necessary components to enable the LiM scheme, elucidating their drawbacks and merits when adapted to noisy and non-ideal programmable resistors. We hope this work will bridge the gap between the device engineers and neural network algorithmists.

Data availability statement

The data that support the findings of this study are available upon reasonable request from the authors.

Acknowledgments

This work is supported by the basic and frontier research project of the Peng Cheng Laboratory, and the major key project of the Peng Cheng Laboratory Grant PCL2023A03.

ORCID iDs

Wei Wang  <https://orcid.org/0000-0002-7206-8659>

Yang Li  <https://orcid.org/0000-0001-9820-0016>

References

- [1] Agarwal S *et al* 2016 Resistive memory device requirements for a neural algorithm accelerator 2016 *Int. Joint Conf. on Neural Networks (IJCNN)* vol 2016 pp 929–38
- [2] Soudry D, Di Castro D, Gal A, Kolodny A and Kvatinsky S 2015 Memristor-based multilayer neural networks with online gradient descent training *IEEE Trans. Neural Netw. Learn. Syst.* **26** 2408–21
- [3] Ielmini D and Wong H-S P 2018 In-memory computing with resistive switching devices *Nat. Electron.* **1** 333–43
- [4] Xia Q and Yang J J 2019 Memristive crossbar arrays for brain-inspired computing *Nat. Mater.* **18** 309–23
- [5] Joshi V, Le Gallo M, Haefeli S, Boybat I, Nandakumar S R, Piveteau C, Dazzi M, Rajendran B, Sebastian A and Eleftheriou E 2020 Accurate deep neural network inference using computational phase-change memory *Nat. Commun.* **11** 2473
- [6] Chen W-H *et al* 2019 CMOS-integrated memristive non-volatile computing-in-memory for AI edge processors *Nat. Electron.* **2** 420–8
- [7] Xue C-X *et al* 2021 A CMOS-integrated compute-in-memory macro based on resistive random-access memory for AI edge devices *Nat. Electron.* **4** 81–90

- [8] Yue J et al 2020 A 65nm computing-in-memory-based CNN processor with 2.9-to-35.8TOPS/W system energy efficiency using dynamic-sparsity performance-scaling architecture and energy-efficient inter/intra-macro data reuse *2020 IEEE Int. Solid- State Circuits Conf.—(ISSCC)* pp 234–6
- [9] Liu Y et al 2016 A 65nm ReRAM-enabled nonvolatile processor with 6× reduction in restore time and 4× higher clock frequency using adaptive data retention and self-write-termination nonvolatile logic *2016 IEEE Int. Solid-State Circuits Conf. (ISSCC)* vol 59 pp 84–86
- [10] Yan B, Li B, Qiao X, Xue C-X, Chang M-F, Chen Y and Li H 2019 Resistive memory-based in-memory computing: from device and large-scale integration system perspectives *Adv. Intell. Syst.* **1** 1900068
- [11] Liu Q et al 2020 A fully integrated analog ReRAM based 78.4TOPS/W compute-in-memory chip with fully parallel MAC computing *2020 IEEE Int. Solid- State Circuits Conf.—(ISSCC)* pp 500–2
- [12] Huo Q et al 2022 A computing-in-memory macro based on three-dimensional resistive random-access memory *Nat. Electron.* **5** 469–77
- [13] Rasch M J et al 2023 Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators *Nat. Commun.* **14** 5282
- [14] Huang S, Jiang H and Yu S 2022 Hardware-aware quantization/mapping strategies for compute-in-memory accelerators *ACM Trans. Des. Autom. Electron. Syst.* **28** 1–23
- [15] Joksas D, Wang E, Barmapsalos N, Ng W H, Kenyon A J, Constantinides G A and Mehonic A 2022 Nonideality-aware training for accurate and robust low-power memristive neural networks *Adv. Sci.* **9** 1–16
- [16] Yao P, Wu H, Gao B, Tang J, Zhang Q, Zhang W, Yang J J and Qian H 2020 Fully hardware-implemented memristor convolutional neural network *Nature* **577** 641–6
- [17] Wan W et al 2022 A compute-in-memory chip based on resistive random-access memory *Nature* **608** 504–12
- [18] Song L, Qian X, Li H and Chen Y 2017 PipeLayer: a pipelined ReRAM-based accelerator for deep learning *2017 IEEE Int. Symp. on High Performance Computer Architecture (HPCA)* pp 541–52
- [19] Li C et al 2018 Efficient and self-adaptive in-situ learning in multilayer memristor neural networks *Nat. Commun.* **9** 2385
- [20] Li C et al 2019 Long short-term memory networks in memristor crossbar arrays *Nat. Mach. Intell.* **1** 49–57
- [21] Wang Z et al 2019 In situ training of feed-forward and recurrent convolutional memristor networks *Nat. Mach. Intell.* **1** 434–42
- [22] He K, Zhang X, Ren S and Sun J 2016 Deep residual learning for image recognition *2016 IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)* vol 45 pp 770–8
- [23] Gao P, Lu J, Li H, Mottaghi R and Kembhavi A 2021 Container: context aggregation network *Advances in Neural Information Processing Systems* vol 23 pp 19160–71
- [24] Silver D et al 2017 Mastering the game of Go without human knowledge *Nature* **550** 354–9
- [25] Silver D et al 2018 A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play *Science* **362** 1140–4
- [26] Amodei D and Hernandez D 2018 AI and compute *Blog Open AI* (available at: <https://blog.openai.com/ai-and-compute/>)
- [27] Mehonic A and Kenyon A J 2022 Brain-inspired computing needs a master plan *Nature* **604** 255–60
- [28] Nagel M, Fournarakis M, Amjad R A, Bondarenko Y, van Baalen M and Blankevoort T 2021 A white paper on neural network quantization (arXiv:2106.08295)
- [29] Gholami A, Kim S, Dong Z, Yao Z, Mahoney M W and Keutzer K 2022 A survey of quantization methods for efficient neural network inference *Low-Power Computer Vision* (Chapman and Hall/CRC) pp 291–326
- [30] Chen Y et al 2015 DaDianNao: a machine-learning supercomputer *Proc. Annu. Int. Symp. Microarchitecture, MICRO* vol 2015 pp 609–22
- [31] Cai F et al 2022 A fully integrated system-on-chip design with scalable resistive random-access memory tile design for analog in-memory computing *Adv. Intell. Syst.* **4** 2200014
- [32] Cai F, Correll J M, Lee S H, Lim Y, Bothra V, Zhang Z, Flynn M P and Lu W D 2019 A fully integrated reprogrammable memristor-CMOS system for efficient multiply-accumulate operations *Nat. Electron.* **2** 290–9
- [33] Yao P et al 2017 Face classification using electronic synapses *Nat. Commun.* **8** 15199
- [34] Kiani F, Yin J, Wang Z, Yang J J and Xia Q 2021 A fully hardware-based memristive multilayer neural network *Sci. Adv.* **7** 1–9
- [35] Ambrogio S et al 2019 Reducing the impact of phase-change memory conductance drift on the inference of large-scale hardware neural networks *2019 IEEE Int. Electron Devices Meeting (IEDM)* pp 6.1.1–4
- [36] Ambrogio S, Balatti S, McCaffrey V, Wang D and Ielmini D 2014 Impact of low-frequency noise on read distributions of resistive switching memory (RRAM) *2014 IEEE International Electron Devices Meeting* vol 2015 pp 14.4.1–4
- [37] Li B, Gu P, Wang Y and Yang H 2016 Exploring the precision limitation for RRAM-based analog approximate computing *IEEE Des. Test* **33** 51–58
- [38] Gokmen T, Onen M and Haensch W 2017 Training deep convolutional neural networks with resistive cross-point devices *Front. Neurosci.* **11** 1–13
- [39] Lillicrap T P, Santoro A, Marris L, Akerman C J and Hinton G 2020 Backpropagation and the brain *Nat. Rev. Neurosci.* **21** 335–46
- [40] Hinton G E, Osindero S and Teh Y-W 2006 A fast learning algorithm for deep belief nets *Neural Comput.* **18** 1527–54
- [41] Hinton G E 2006 Reducing the dimensionality of data with neural networks *Science* **313** 504–7
- [42] Wang W et al 2022 Efficient training of the memristive deep belief net immune to non-idealities of the synaptic devices *Adv. Intell. Syst.* **4** 2100249
- [43] Lillicrap T P, Cownden D, Tweed D B and Akerman C J 2016 Random synaptic feedback weights support error backpropagation for deep learning *Nat. Commun.* **7** 13276
- [44] Agarwal S et al 2017 Achieving ideal accuracies in analog neuromorphic computing using periodic carry *2017 Symp. on VLSI Technology* pp T174–5
- [45] Frascaroli J, Brivio S, Covi E and Spiga S 2018 Evidence of soft bound behaviour in analogue memristive devices for neuromorphic computing *Sci. Rep.* **8** 7178
- [46] Chen P-Y, Peng X and Yu S 2017 NeuroSim+: an integrated device-to-algorithm framework for benchmarking synaptic devices and array architectures *2017 IEEE Int. Electron Devices Meeting (IEDM)* pp 6.1.1–4
- [47] Ernoul M, Grollier J and Querlioz D 2019 Using memristors for robust local learning of hardware restricted boltzmann machines *Sci. Rep.* **9** 1851
- [48] Prezioso M, Merrih-Bayat F, Hoskins B D, Adam G C, Likharev K K and Strukov D B 2015 Training and operation of an integrated neuromorphic network based on metal-oxide memristors *Nature* **521** 61–64

- [49] Danial L, Pikhay E, Herbelin E, Wainstein N, Gupta V, Wald N, Roizin Y, Daniel R and Kvatinsky S 2019 Two-terminal floating-gate transistors with a low-power memristive operation mode for analogue neuromorphic computing *Nat. Electron.* **2** 596–605
- [50] Gokmen T and Vlasov Y 2016 Acceleration of deep neural network training with resistive cross-point devices: design considerations *Front. Neurosci.* **10** 333
- [51] Nandakumar S R et al 2020 Mixed-precision deep learning based on computational memory *Front. Neurosci.* **14** 406
- [52] Schiffmann W, Joost M and Werner R 1994 *Optimization of the Backpropagation Algorithm for Training Multilayer Perceptrons* (Springer)
- [53] Zhang W et al 2023 Edge learning using a fully integrated neuro-inspired memristor chip *Science* **381** 1205–11
- [54] Burr G W et al 2015 Experimental demonstration and tolerancing of a large-scale neural network (165 000 synapses) using phase-change memory as the synaptic weight element *IEEE Trans. Electron Devices* **62** 3498–507
- [55] Chen P, Liu F, Lin P, Li P, Xiao Y, Zhang B and Pan G 2023 Open-loop analog programmable electrochemical memory array *Nat. Commun.* **14** 6184
- [56] Alibart F, Zamanidoost E and Strukov D B 2013 Pattern classification by memristive crossbar circuits using *ex situ* and *in situ* training *Nat. Commun.* **4** 2072
- [57] Alibart F, Gao L, Hoskins B D and Strukov D B 2012 High precision tuning of state for memristive devices by adaptable variation-tolerant algorithm *Nanotechnology* **23** 075201
- [58] Ambrogio S et al 2018 Equivalent-accuracy accelerated neural- network training using analogue memory *Nature* **558** 60–67
- [59] Mackin C, Tsai H, Ambrogio S, Narayanan P, Chen A and Burr G W 2019 Weight programming in DNN analog hardware accelerators in the presence of NVM variability *Adv. Electron. Mater.* **5** 1900026
- [60] Rasch M J et al 2021 A flexible and fast PyTorch toolkit for simulating training and inference on analog crossbar arrays 2021 *IEEE 3rd Int. Conf. Artif. Intell. Circuits Syst. AICAS 2021* pp 1–11
- [61] Gokmen T and Haensch W 2020 Algorithm for training neural networks on resistive device arrays *Front. Neurosci.* **14** 103
- [62] Gokmen T 2021 Enabling training of neural networks on noisy hardware *Front. Artif. Intell.* **4** 1–14
- [63] Li Y et al 2024 Binary-stochasticity-enabled highly efficient neuromorphic deep learning achieves better-than-software accuracy *Adv. Intell. Syst.* **6** 2300399
- [64] Wang W, Danial L, Li Y, Herbelin E, Pikhay E, Roizin Y, Hoffer B, Wang Z and Kvatinsky S 2022 A memristive deep belief neural network based on silicon synapses *Nat. Electron.* **5** 870–80